

Design and Analysis of Algorithms

- Introduction
- Algorithm
- Max in an Array and Insertion sort
- Loop Invariant

Yanlin Zhang & Shangqi Lu | DSAA 2043 Fall 2025

- Lecture Time: Mon + Wed, 09:00 – 10:20am, E1-101
- Lab Session: Fri, 12:00 – 12:50pm, E1-101 → E1-227
- Instructor: Yanlin Zhang
 - yanlinzhang AT hkust-gz.edu.cn
 - <https://zhyanlin.github.io/>
- Office hour: Thursday 10-11 am, E1-5F-511
- TA
 - Junning Feng, jfeng496@connect.hkust-gz.edu.cn
 - Houcheng Su, hsu638@connect.hkust-gz.edu.cn
- Course materials
 - Canvas

Introduction

- The **design and analysis** of algorithms
 - They usually appear together
- By taking this course, you will
 - Obtain a good understanding of various **data structures and algorithms**
 - Learn to **think analytically** about algorithms
 - Learn to **design and apply algorithms** to solve computational problems **effectively**
 - Learn to **implement and evaluate** algorithms and data structures

Contents (Tentative)

Week	Topic	Content(if any)
1	Course introduction & basic algorithms	RAM model & Array
2	Analysis of algorithms	Asymptotic Analysis
3	Sorting algorithms	
4	Divide-and-conquer	Merge sort, Binary search, Integer multiplication, Master's theorem
5	Basic data structures	
6	Advanced data structures	
7	Dynamic Programming I	Knapsack Problem
8	Dynamic Programming II and Midterm Exam	Longest Common Subsequence, backtracking
9	Greedy	Scheduling, MST
10	Graph algorithm I	Graph representation, Graph traversal, Topological sorting, Cycle detection
11	Graph algorithm II	Strongly connected component, etc.
12	Graph algorithm III	shortest path algorithms
13	NP	P/NP, NP-complete
14		

Textbook:

- [Introduction to Algorithms](#). Cormen, Leiserson, Rivest, and Stein (CLRS)

Reference books:

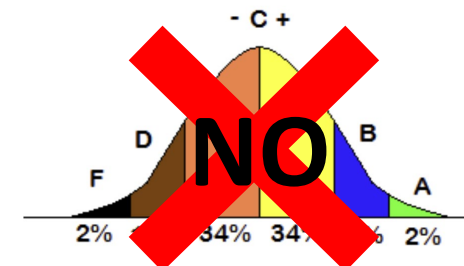
- [Algorithm Design](#). Kleinberg and Tardos
- [The Algorithm Design Manual](#). Steven Skiena

Online resources:

- MIT 6.006 - [Introduction to Algorithms](#)
- Stanford CS161 - [Design and Analysis of Algorithms](#)

Assessment and Grading (Tentative)

- Class Participation (5%)
- Lab Exercises and OJ (10%+5%):
work on lab exercises and submit by the deadline (each week);
Solving 30 OJ problems
- Individual Project (15%):
a two-phase programming exercise
- Mid-term exam (25%):
closed book, on computer, week 8
- Final exam (40%):
closed book, written
- We assess student performance using **criterion-referencing** approach.
In addition to the criterion written in course syllabus, you can **estimate your performance** from your course work score:
 - A level: [85, 100] – C level: [55, 70] – F level: [0, 40]
 - B level: [70, 85) – D level: [40, 55)



How to get the most out of this course

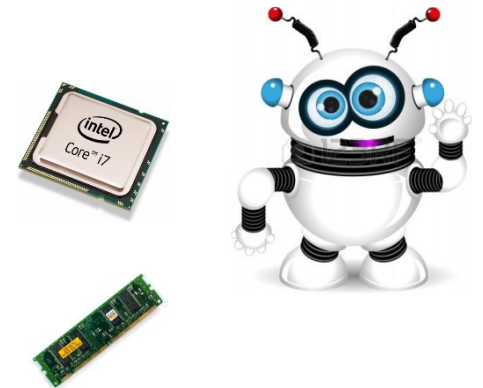
Preview, Participate, and Review matters!

- **Before class:**
 - Prepare for the lecture
- **During class:**
 - Class participation: ask any questions anytime
 - Engage with in-class questions and exercises
- **After class:**
 - Review contents timely and ask questions 😊 → Don't wait until the day before exam 😞
 - Do exercises
- **Generative AI:**
 - Using Generative AI to prepare and review course content is allowed.
 - Don't use it (brainlessly) to solve exercise.
 - Learning requires **generation** by you (not AI)
 - Learning algorithm do require learning **abstraction**, **in-depth thinking**, and **asking critical questions!** 8

- **Time: 13:00–14:30, Sat, Nov 1, 2025**
- Location: TBA
- Format: On computer (solve algorithmic problems by writing code)
- Preparation:
 - Having a good understanding of algorithms covered in class
 - Learn to write code without LLM (i.e., no copilot, no cursor, etc.)
 - Know basic python grammar such as use *self* in a python class.
- Fun data of last course offering:
 - Mean/median midterm score: ~50 → many students failed at writing bug-free code
 - Mean/median course score: ~80

Algorithm

- Natural Languages
 - Chinese, English, Japanese, ...
- **Programming Languages (PLs)**
 - High Level PLs
 - Pascal, C, Java, ...
 - Low Level PLs
 - Assembly Languages
 - A Machine Language executed by a CPU
- **Programming: to tell what CPU to do step-by-step**
 - $2 + 3 = ?$
 - Input/Output,
 - Get the answer (computing)

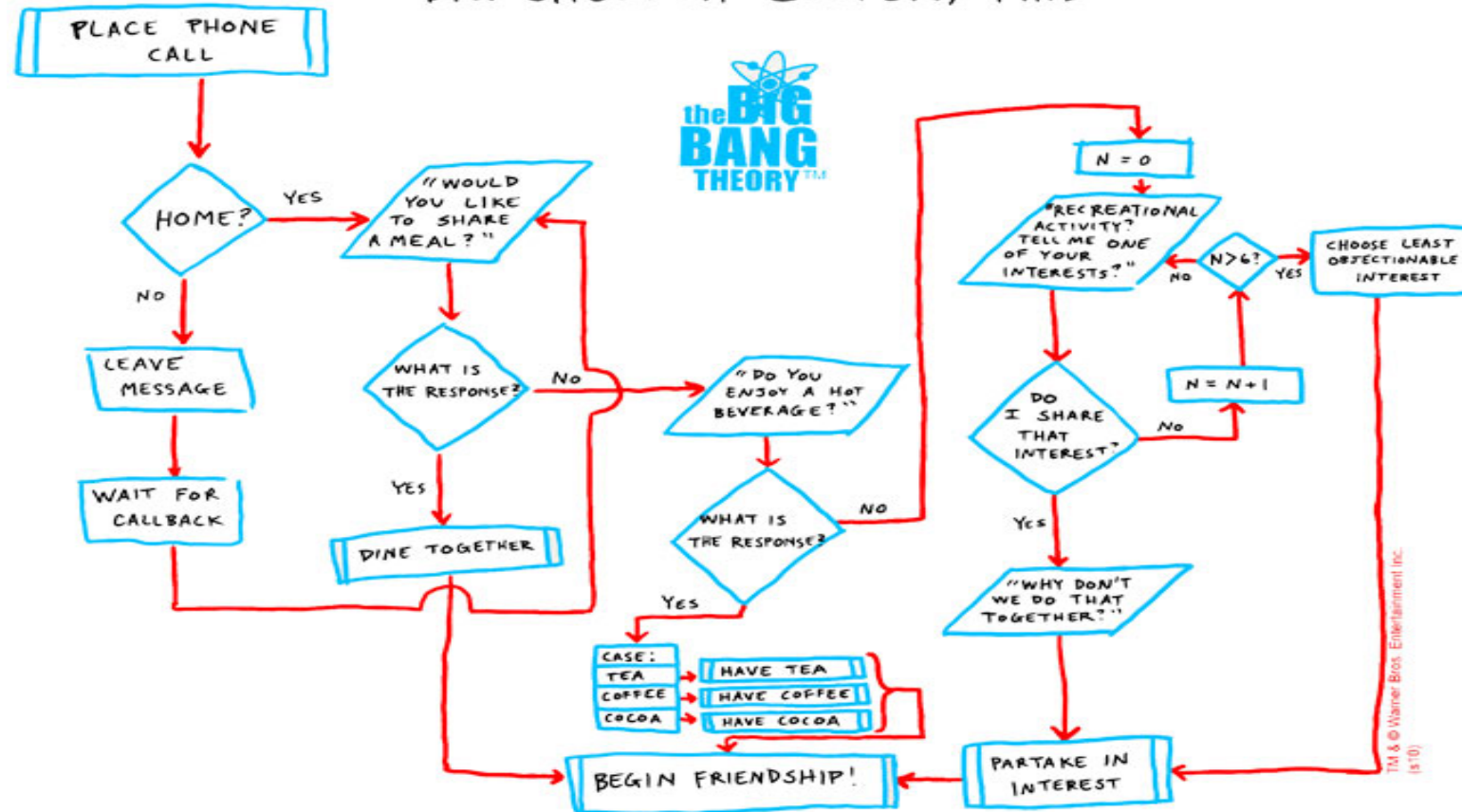


- **Algorithm:** Outline, the essence of a computational procedure, step-by-step instructions
- **Program:** an implementation of an algorithm in some programming language
- **Data structure:** Organization of data needed to solve the problem

What Is an Algorithm

THE FRIENDSHIP ALGORITHM

DR. SHELDON COOPER, Ph.D



An Algorithm is Like a Recipe

- The ingredients
- The equipment
- The list of steps



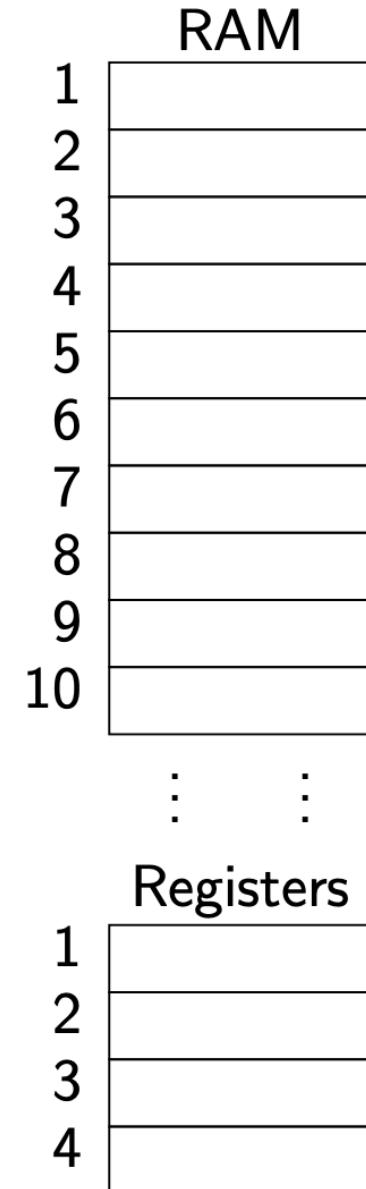
- An **algorithm** is a **finite set of instructions** that, if followed, accomplishes a particular task to solve a problem.
- All algorithms satisfy the following criteria:
 - **Input**: 0 or more quantities are supplied.
 - **Output**: At least one quantity is produced.
 - **Definiteness**: Each instruction is clear and unambiguous.
 - **Finiteness**: For all cases, the algorithm terminates after a finite number of steps.
 - **Effectiveness**: Every instruction must be basic enough (feasible).

- Unlike human, one instruction can only do a very basic thing,
 - read/write a single data value
 - compare two data values
 - Arithmetic operations (+, -, $\times$, $\div$), etc.
 - **Depends on computer, programming language, ...**
- Consider sorting cards.
 - Human can quickly sort cards in order, because they can see all the cards simultaneously.
 - An algorithm cannot.



- **Real Computers are complicated**
 - Memory hierarchy, floating point operations, garbage collector, compiler optimizations, different programming languages, . . .
- **Models of Computation:**
 - Simple abstraction of a Computer
 - Defines the “Rules of the Game”:
 - Which operations is an algorithm allowed to do?
 - What is the cost of each operation?
 - Cost of an algorithm = $\sum$ cost of all its operations

- **RAM Model:** Random Access Machine Model
 - Infinite Random Access Memory (an array), each cell has a unique address
 - Each cell stores one word, e.g., an integer, a character, an address, etc.
 - **Input:** Stored in RAM
 - **Output:** To be written into RAM
 - A finite (constant) number of registers (e.g., 4)
- **In a single Step we can:**
 - Load a word from memory into a register
 - Compute (+, −, *, /), bit operations, comparisons, etc. on registers
 - Move a word from register to memory



- **Algorithm in the RAM Model**

Sequence of elementary operations (similar to assembler code)

- **Example:** Compute the sum of two integers

Assume that $M[1]$ and $M[2]$ contain the integers

Write output to position $M[3]$

- **Cost of an Algorithm:**

Runtime: Total cost of all elementary operations

Space: Total number of memory cells used

- This is not a programming course; however, you will engage in exercises that focus on problem-solving through programming.
- Programming is the last step in software development, which occurs only after the algorithms are clear.
- Computer science is a branch of mathematics with its art reflected in the beauty of algorithms.
 - Programming knowledge is not necessary to study algorithms.

Max in an Array and Insertion sort

- An **array** is a **linear data structure** that stores a **fixed-size** collection of elements of the **same type** in **contiguous memory locations**.
 - **Fixed Size** – Its size is **declared at initialization** and cannot be changed.
 - **Contiguous Memory Allocation** – Elements are **stored sequentially** in memory, making **access fast** ($O(1)$ for direct access by index).
 - **Homogeneous** – All elements in an array must be of the same data type.
 - **Index-Based Access** – Elements are **accessed using an index**, starting from 1.

Array:

10	25	30	40	50
----	----	----	----	----

Index:

1 2 3 4 5

Max in an Array



Algorithm findMax(A)



Input: An array A storing n values



Output: The maximum element in A

- **Pseudo-code:** A mixture of natural language and high-level programming concepts that describes the main ideas behind a generic implementation of a data structure or algorithm.
- It is more structured than usual prose but less formal than a programming language
- Expressions
 - use standard mathematical symbols to describe numeric and boolean expressions
 - use $\leftarrow$ for assignment (“=” in Python)
 - use = for equality relationship (“==” in Python)
- Method declarations
 - algorithm name(param1,param2)

- Programming constructs
 - decision structures: if ... then ... [else ...]
 - while-loops: while ... do
 - repeat-loops: **repeat ... until ...**
 - for-loop: for ... do Subarray includes A[j]
 - array indexing: A[i], A[i:j], A[i,j] or A[i][j]
array index starts from 1 2d array
- Methods
 - calls: object method(args)
 - returns: **return** value

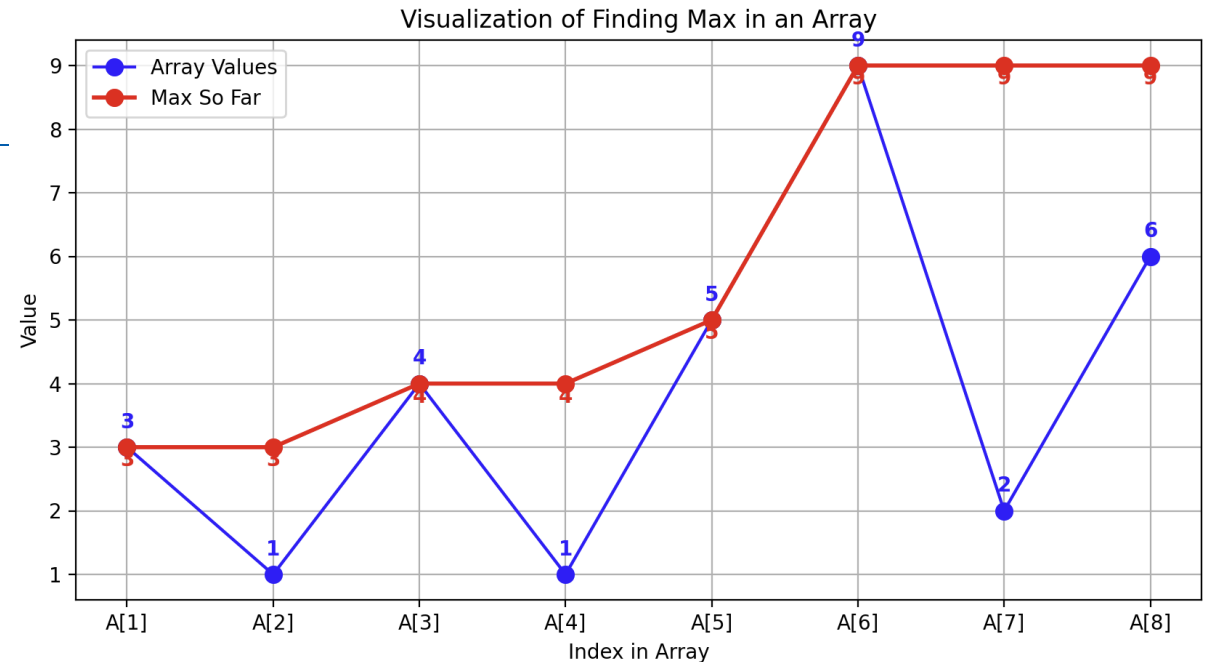
findMax pseudo-code

Algorithm FindMax(A)

Input: An array A with n elements

Output: The maximum value in A

```
1. max_so_far ← A[1]  // Initialize max with the first element
2. for i ← 2 to length(A) do
3.   if A[i] > max_so_far then
4.     max_so_far ← A[i]  // Update max if a larger value is found
5.   end if
6. end for
7. return max_so_far
```





Algorithm $\text{sort}(A)$



Input: An array A storing n values



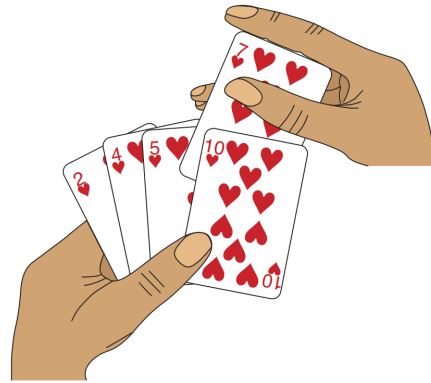
Output: A permutation of A where elements are ordered in increasing sequence

INPUT

sequence of numbers

$a_1, a_2, \dots, a_n$

2 5 4 10 7



OUTPUT

a permutation of the
sequence of numbers

$b_1, b_2, \dots, b_n$

2 4 5 7 10

Correctness (requirements for the output)

For any given input the algorithm halts
with the output:

- $b_1 < b_2 < b_3 < \dots < b_n$
- $b_1, b_2, b_3, \dots, b_n$ is a permutation of
 $a_1, a_2, a_3, \dots, a_n$

Running time

Depends on

- number of elements (n)
- how (partially) sorted
they are
- algorithm

Picking and Placing Cards at Hand

Poker-Style Insertion Sort (magic power)

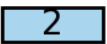
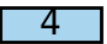
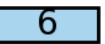
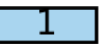
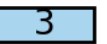
Table:      
Hand:

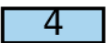
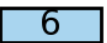
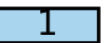
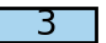
Table:     
Hand: 

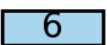
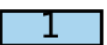
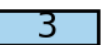
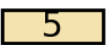
Table:    
Hand:  

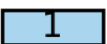
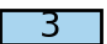
Table:   
Hand:   

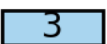
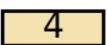
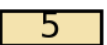
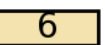
Table:  
Hand:    

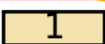
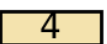
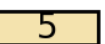
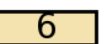
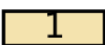
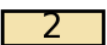
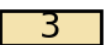
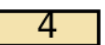
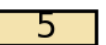
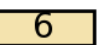
Table:  
Hand:     

Table:
Hand:      

Strategy

- Start “empty handed”
- Insert a card in the right position of the already sorted hand
- Continue until all cards are inserted/sorted

Poker-Style Insertion Sort (no magic power)

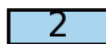
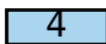
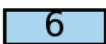
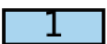
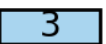
Table:      
Hand:

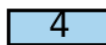
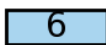
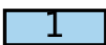
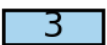
Table:     
Hand: 

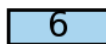
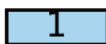
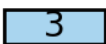
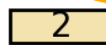
Table:    
Hand:  

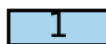
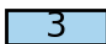
Table:   
Hand:   

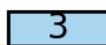
Table:  
Hand:    

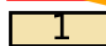
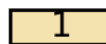
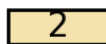
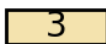
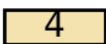
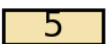
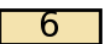
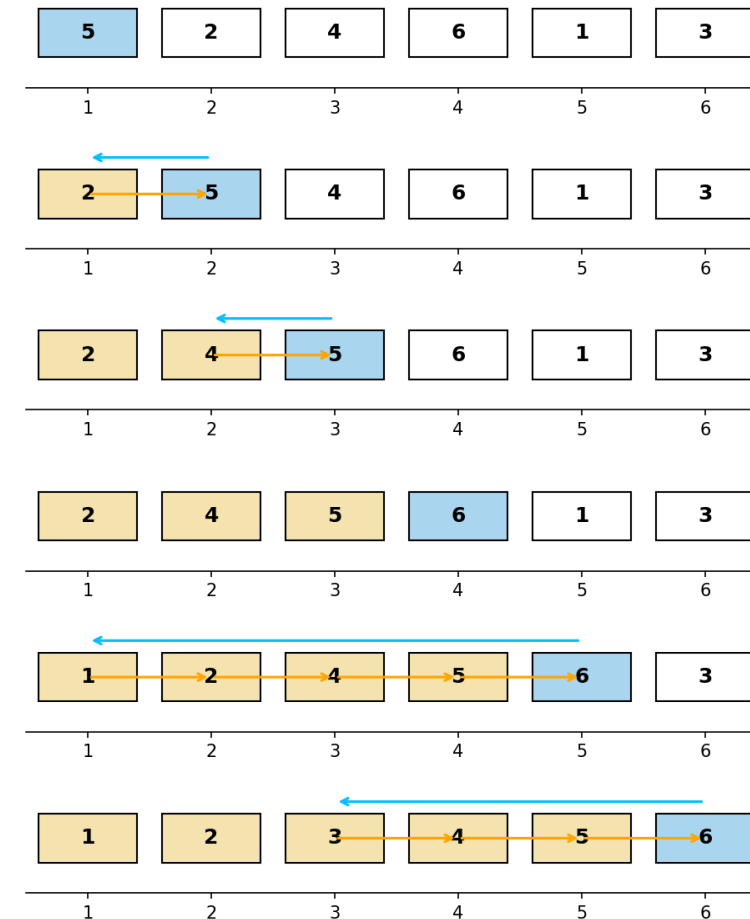
Table:  
Hand:     

Table:
Hand:      

Insertion Sort

Insertion Sort Step-by-Step



Algorithm InsertionSort(A)

Input: An array A with n elements

Output: A sorted in non-decreasing order

```
1. for i ← 2 to length(A) do
2.     key ← A[i] // Current element to be inserted
3.     j ← i - 1 // Start comparing from the previous element
4.     while j > 0 and A[j] > key do
5.         A[j + 1] ← A[j] // Shift element to the right
6.         j ← j - 1
7.     end while
8.     A[j + 1] ← key // Place key in the correct position
9. end for
10. return A // Sorted array
```

Is our algo. correct?

Loop Invariant

Algorithm FindMax(A)

Input: An array A with n elements

Output: The maximum value in A

```
1. max_so_far ← A[1]
2. for i ← 2 to length(A) - 1 do
3.     if A[i] > max_so_far then
4.         max_so_far ← A[i]
5.     end if
6. end for
7. return max_so_far
```

Proof of Correctness

Goal: Showing that max_so_far stores the maximum value of A.

1. Before the Loop Starts (Initialization)

- Before entering the loop, max_so_far = A[1]. Since we've only seen one element, this is correct.

2. During Each Iteration (Maintenance)

- If $A[i] > \text{max_so_far}$, we update $\text{max_so_far} = A[i]$, ensuring it holds the maximum seen so far.
- Otherwise, max_so_far remains unchanged, which is still correct.

3. After the Loop Ends (Termination)

- At the end, max_so_far contains the maximum of all A[1] to A[n].

- A **loop invariant** is a property or condition that holds true before and after each iteration of a loop. **within the loop, 不一定对**
- Purpose:
 - To show that an algorithm maintains a specific condition throughout its execution.
 - To help prove that the algorithm works correctly (via **initialization, maintenance, and termination**).
- The way that we prove FindMax correct is Loop Invariant.
 - Loop invariant: `max_so_far` holds the maximum value among `A[1:i]`.

Structure of a Loop Invariant Proof

- Initialization: Show that the invariant holds **before** the first iteration (base case).
- Maintenance: Assuming the invariant holds at the beginning of **any** iteration, **prove that it still holds** after executing the loop body.
- Termination: When the loop terminates, the invariant (plus the loop's exit condition) gives a useful property that helps prove the algorithm's correctness.

*FYI. The three steps is introduced in **CLRS**. In other books, you may find Establishment (i.e. Initialization), Preservation (i.e. Maintenance), Postcondition and Termination: Postcondition ensures the final goal is achieved if the loop stops; Termination guarantees that the loop will stop.*

Proving Insertion-sort

Algorithm InsertionSort(A)

Input: An array A with n elements

Output: A sorted in non-decreasing order

```
1. for i ← 2 to length(A) do
2.     key ← A[i]
3.     j ← i - 1
4.     while j > 0 and A[j] > key do
5.         A[j + 1] ← A[j]
6.         j ← j - 1
7.     end while
8.     A[j + 1] ← key
9. end for
10. return A
```

Proof of Correctness

Loop Invariant: $A[1:i-1]$ is a sorted list of elements in the original $A[1:i-1]$

1. Initialization (Note: $i \leftarrow 2$ is not part of the loop)

- When $i=2$, $A[1:1]$ contains one value $\rightarrow$ sorted.

2. Maintenance

- Within the loop-body, $A[1:i-1]$ is assumed to be sorted. We move $A[i-1]$, $A[i-2]$, ... toward the right, until we find a position for $A[i]$. Once $A[i]$ is inserted, $A[1:i]$ remains sorted.

3. Termination

- As i goes from 2 to $\text{length}(A)$, and the loop body does not modify i . The loop will terminate. By termination, $i=\text{length}(A)$ means $A[1:\text{length}(A)]$ is sorted.

Discussion: a Loop Invariant for Linear Search

Algorithm LinearSearch(A, x)

Input: An array A with n elements, target value x

Output: Index of x in A, or -1 if not found

```
1. loc = -1
2. for i ← 1 to length(A) do
3.     if A[i] == x then
4.         loc = i // Found x at index i
5.     end if
6. end for
7. return loc
```

0. loc为x在A[1:i-1]中最后一次出现的索引, 无则-1

1. i=1, i-1=0, A[1:0]空, loc=-1

2. 分if成立/不成立讨论

3. 结束时A[1:n], 正是整个数组A

Proof of Correctness

Loop Invariant: What is true and related to this task?

1. Initialization: What is true about A[1:i] before the loop starts?

2. Maintenance: How does each iteration preserve the correctness of the search?

3. Termination: When the loop exits, why can we be sure the correct index or -1 is returned?

Using the **Loop Invariant Proof Structure**, prove that LinearSearch(A, x) is correct.

Discussion: a Loop Invariant for Linear Search

Algorithm LinearSearch2(A, x)

Input: An array A with n elements, target value x

Output: Index of x in A, or -1 if not found

```
1. for i ← 1 to length(A) do
2.     if A[i] == x then
3.         return i // Found x at index i
4.     end if
5. end for
6. return -1 // x is not in A
```

在每次迭代开始时, 元素x不存在于已检查过的数组A[1:i-1]中

Proof of Correctness

Loop Invariant:

1. Initialization:

2. Maintenance:

3. Termination:

Using the [Loop Invariant Proof Structure](#), prove that LinearSearch2(A, x) is correct.

Finding the Sugared Water

🤔 You have **100 cups of water**, but **only one** contains sugar. How can you find it quickly?

- Possible Solution: Naïve Approach (Brute Force)
 - 😞 Taste each cup one by one → **100 checks** needed
- Can We Do Better?
 - Is there a **smarter, faster** way?



An Example

- Suppose we use a **list** of integers as the data structure to store a set S of n integers.
- Consider how to support the **search operation**:
 - A function $\text{search}(S, \text{search_num})$ that returns the index i of search_num in the list, or -1 if it is not found.
- Example:
 - $S = \{3, 7, 9, 12, 13, 18, 20, 23, 27\}$
 - Is 20 in S ?
- We give two algorithms to implement it.

An Example: Linear Search

- Simply read the integer of index i for each $i \in [0, n - 1]$.
- If there exists i s.t. $S[i] = \text{search_num}$, return i ;
- Otherwise, return -1.

```
def search(integer_set, search_num):  
    for i in range(len(integer_set)):  
        if integer_set[i] == search_num:  
            return i # Return the index if found  
    return -1 # Return -1 if not found
```

An Example: Binary Search

- Assume sets of integers are sorted. 对于二分查找，数组必须是排好序的
- Let us compare search_num to the element in the middle (the $(n/2)$ -th) of S)

```
def binary_search(sorted_set, search_num):  
    left = 0  
    right = len(sorted_set) - 1  
  
    while left <= right:  
        middle = (left + right) // 2 # Find the middle index  
  
        if sorted_set[middle] < search_num:  
            left = middle + 1 # Narrow search to the right half  
        elif sorted_set[middle] > search_num:  
            right = middle - 1 # Narrow search to the left half  
        else:  
            return middle # Return the index if found  
  
    return -1 # Return -1 if not found
```

searchnum = 18 size = 9

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

left=0 right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=4

left=5 right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=6

left=5 right=5

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=5

```
searchnum = 28    size = 9
```

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

left=0

right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=4

left=5

right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=6

left=7 right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=7

left=8 right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=8

Linear search VS binary search: which one is better?

```
def search(integer_set, search_num):  
    for i in range(len(integer_set)):  
        if integer_set[i] == search_num:  
            return i # Return the index if found  
    return -1 # Return -1 if not found
```

```
def binary_search(sorted_set, search_num):  
    left = 0  
    right = len(sorted_set) - 1  
  
    while left <= right:  
        middle = (left + right) // 2 # Find the middle index  
  
        if sorted_set[middle] < search_num:  
            left = middle + 1 # Narrow search to the right half  
        elif sorted_set[middle] > search_num:  
            right = middle - 1 # Narrow search to the left half  
        else:  
            return middle # Return the index if found  
  
    return -1 # Return -1 if not found
```

$O(\log n)$